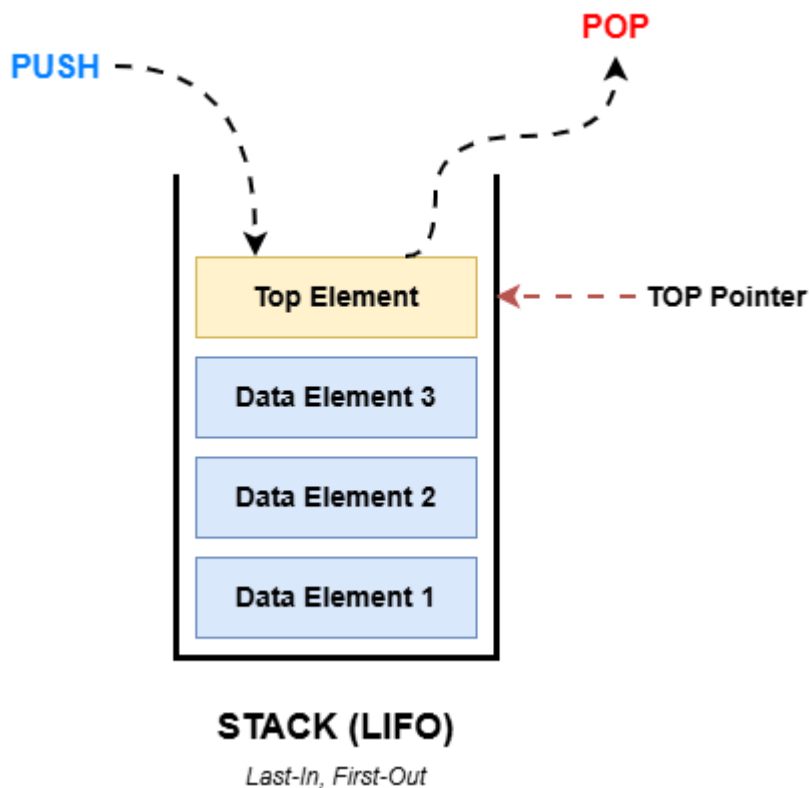


Stack data Structure

Stack

1. Linear data structure.
2. that works on LIFO principle



operations are:

1. Push (Insert Element):- Add element to the top of stack.

```
stack = []  
stack.append(10)  
stack.append(20)  
stack.append(30)  
print(stack)
```

O/P:- [10, 20, 30]-

2. Pop(Remove Element):- Remove top element.

```
stack.pop()  
print(stack)
```

```
# O/P:- [10,20]
```

3. isEmpty():- Check stack empty or not.

```
if len(stack) == 0:
    print("Stack is empty")
else:
    print("Stack is not empty")
```

4. isfull() :- Check stack full or not.

```
if len(stack) == 10:
    print("Stack is full")
else:
    print("Stack is not full")
```

5. Peek / Top:- See top element without removing.

```
print(stack[-1])
```

```
# O/P:- 20
```

stack with functions:-

```
def stack():
    stack = []
    return stack

def push(stack, item):
    stack.append(item)

def pop(stack):
    if len(stack) == 0:
        raise IndexError("Stack Underflow")
    return stack.pop()

def top(stack):
    if not stack:
        print("Stack is empty. Cannot peek an item.")
        return None
    return stack[-1]

def isEmpty(stack):
    if len(stack) == 0:
        return True
```

```

        return False

def isFull(stack, capacity):
    if len(stack) == capacity:
        return True
    return False

stack = stack()
item = eval(input("Enter an item to push onto the stack: "))
push(stack, item)

popped_item = pop(stack)
print(f"Popped item: {popped_item}")

top_item = top(stack)
print(f"Top item: {top_item}")

isEmpty_result = isEmpty(stack)
print(f"Is the stack empty? {isEmpty_result}")

capacity = eval(input("Enter the capacity of the stack: "))
isFull_result = isFull(stack, capacity)
print(f"Is the stack full? {isFull_result}")

```

stack with class

```

class Stack:
    def __init__(self, capacity=None):
        self.stack = []
        self.capacity = capacity

    def push(self, item):
        if self.capacity is not None and len(self.stack) == self.capacity:
            print("Stack is full. Cannot push an item.")
            return
        self.stack.append(item)

    def pop(self):
        if not self.stack:

```

```

        print("Stack is underflow. Cannot pop an item.")
        return None
    return self.stack.pop()

def top(self):
    if not self.stack:
        print("Stack is empty. Cannot peek an item.")
        return None
    return self.stack[-1]

def isEmpty(self):
    return len(self.stack) == 0

def isFull(self):
    if self.capacity is None:
        return False
    return len(self.stack) == self.capacity

# ---- Usage ----
capacity = int(input("Enter the capacity of the stack: "))
s = Stack(capacity)

item = eval(input("Enter an item to push onto the stack: "))
s.push(item)

popped_item = s.pop()
print(f"Popped item: {popped_item}")

top_item = s.top()
print(f"Top item: {top_item}")

print(f"Is the stack empty? {s.isEmpty()}")
print(f"Is the stack full? {s.isFull()}")

```